COP3530 Exam 2 Cheat Sheet Modules 8–12

SORTING

Complexities

Sort	Best	Avg	Worst	Space	IP?
Bubble	n	n^2	n^2	1	✓
Selection	n^2	n^2	n^2	1	×
Insertion	n	n^2	n^2	1	✓
Shell	$n^{7/6}$	$n^{5/4}$	n^2	1	×
Quick	$n \lg n$	$n \lg n$	n^2	$\lg n$	×
Merge	$n \lg n$	$n \lg n$	$n \lg n$	n	×
Heap	$n \lg n$	$n \lg n$	$n \lg n$	1	×

In-Progress Rules

- **Bubble**: after k passes, **last** k elements are final
- **Selection**: after k passes, **first** k elements are final
- **Insertion**: after k passes, **first** $k+1$ elements are sorted (relative order correct, positions may shift later)
- **Quick** (1 pass): pivot is in its **final** position; left subarray $\leq$ pivot, right $>$ pivot — neither side sorted yet
- **Merge**: subarrays of size 2^k are sorted after k levels; final merge produces sorted array
- **Heap** (build): after heapify, valid max-heap; each extraction places max at the end

Algorithm Sketches

Quick sort — pivot = arr[0]. Move `up` ptr right until > pivot; move `down` ptr left until $\leq$ pivot; swap. When `down` crosses `up`, swap pivot with `arr[down]`.

Merge sort — split → recurse left & right → merge two sorted halves into temp array → copy back.

Heap sort — build max-heap $O(n)$, then repeatedly swap root with last unsorted element and heapify $O(\log n)$. Total: $O(n \log n)$.

SETS, MAPS & HASHING

Set vs Map

- **set** — unique elements only; no index access
- **map** — unique *key*/value pairs; `map[k]=v` overwrites
- `std::set/map` ⇒ BST, $O(\log n)$
- `std::unordered_set/map` ⇒ hash table, $O(1)$ avg

Key Operations

`insert`, `erase`, `find` (returns iterator or `end()`), `count` (0 or 1), `size`, `empty`

Unordered extras: `bucket_count()`, `load_factor()`

Hash Tables

Load factor $\lambda = \frac{\#\text{ elements}}{\#\text{ buckets}}$

Rehash when λ too large (slow chaining/clustering).

Good hash: deterministic, fast, uniform distribution, minimizes collisions.

Collision Resolution

1. **Separate Chaining** — each bucket is a linked list; collision appended to list
2. **Linear Probing** — collision: probe **+1** bucket at a time until empty slot found
3. **Quadratic Probing** — collision: probe offsets $1^2, 2^2, 3^2, \ldots$ (i.e. $+1, +4, +9, \ldots$)

Probing example (table size 7, hash = key % 7):

Insert 10 (slot 3), insert 17 (slot 3 → **Linear**: try 4; **Quad**: try 3+1=4)

GRAPHS & TRAVERSALS

Terminology

- **Adjacent**: B is adjacent to A iff edge $A \to B$ exists (directed: one-way!)
- The *adjacency set of* A contains B; B's does **not** contain A (unless separate edge)
- **Path**: sequence of adjacent vertices
- **Simple path**: no repeated vertices (first = last ⇒ cycle)
- **Cycle**: simple path where first = last vertex
- **In-degree**: edges *into* a vertex; **Out-degree**: edges *out of*
- **Dense**: $|E| \approx |V|^2$; **Sparse**: $|E| \approx |V|$
- **Simple Graph**: **no self-loops** and **no parallel edges** (at most one edge from any vertex u to any vertex v)

Graph Types

Directed/Undirected (arrow direction matters) **Weighted/Unweighted** **Connected/Disconnected** **Cyclic/Acyclic**

Real examples: Facebook → undirected unweighted; Web → directed; Flights → directed weighted cyclic

Representations

Impl.	Connectedness	Adjacency	Space
Edge List	$O(E)$	$O(E)$	$O(E)$
Adj. Matrix	$O(1)$	$O(V)$	$O(V^2)$
Adj. List	$O(V)$	$O(V)$	$O(V+E)$

- **Adj. Matrix**: `graph[from][to]=weight`; best for dense graphs
- **Adj. List**: `map<from, vector<pair<to,wt>>>`; best for sparse

BFS & DFS

BFS (queue) — enqueue start; while queue non-empty: dequeue u, enqueue all unvisited neighbors of u.

Use case: shortest path on **unweighted** graph only. **Does NOT work for weighted graphs** — use Dijkstra's instead. Output (ex): A, B, C, D, E, F

DFS (stack) — push start; while stack non-empty: peek u; if u has unvisited neighbor v, push v; else pop u.

Use case: topological sort, cycle detection. Output (ex): A, B, D, C, E, F

S-T Path — run BFS/DFS from s; return true if t visited.

BFS & DFS Complexities

	Time	Space
BFS (iterative)	$O(V+E)$	$O(V)$
DFS (iterative)	$O(V+E)$	$O(V)$
DFS (recursive)	$O(V+E)$	$O(V)$

Time $O(V+E)$: visit every vertex once ($O(V)$) + traverse every edge once ($O(E)$).

Space $O(V)$: queue/stack/call-stack holds at most V vertices.

Note: recursive DFS space = $O(V)$ call-stack frames in worst case (a path graph); iterative DFS uses explicit stack of same size. Both implementations identical in complexity.

GRAPH ALGORITHMS

Dijkstra's (Shortest Weighted Path)

Input: directed graph, non-negative weights, source s

Output: shortest distance from s to every vertex

Complexity: $O(V \log V + E \log V)$

Greedy — always correct if all weights ≥ 0

Steps:

1. Init: `dist[s]=0`, `dist[v]=`∞, `pred[v]=null`, *Done*$=\emptyset$
2. **Repeat** until *NotDone* empty:
   - Pick v = min `dist` not in *Done*
   - Add v to *Done*
   - **Relax** all edges from v:

```
if dist[to] > dist[v] + w:
dist[to] = dist[v] + w;
pred[to] = v
```

Reading result: trace `pred[]` backwards from destination

Edge weight $D \to C$ = `dist[C]` − `dist[D]` (when D immediately precedes C)

Minimum Spanning Trees

Spanning tree: subset of edges connecting all V vertices with no cycles. Has $V-1$ edges. **MST**: spanning tree with minimum total edge weight.

Prim's Algorithm:

1. Start with any vertex in set S
2. Repeatedly: find min-weight edge crossing the *cut* (S vs $V-S$), add its endpoint to S
3. Until $V-S$ is empty

Cut property: the lightest edge crossing any cut is always safe to add.

Kruskal's Algorithm:

1. Sort all edges by weight (ascending)
2. For each edge: add to MST if it does **not** create a cycle

3. Use **Disjoint Set / Union-Find** to detect cycles efficiently:
   - `find(x)`: returns root of x's component
   - `union(x,y)`: merge components of x and y
   - Cycle detected if `find(u) == find(v)` before adding edge $u{-}v$

Topological Sort

Applies to: DAGs (directed acyclic graphs) only.

Definition: ordering where if edge $V_i \to V_j$ exists, then V_i comes before V_j.

Steps (Kahn's / BFS-based):

1. Compute in-degree of every vertex
2. Enqueue all vertices with in-degree $= 0$
3. While queue non-empty:
   Dequeue v, output v
   For each neighbor u of v: decrement in-degree$[u]$
   If in-degree$[u] = 0$, enqueue u

GREEDY ALGORITHMS

Huffman Coding

Goal: lossless compression — assign shorter codes to more frequent characters.

Property: prefix-free (no code is a prefix of another).

Build Huffman Tree:

1. Create a leaf node for each symbol with its frequency
2. Insert all nodes into a **min-heap** (priority queue by frequency)
3. While heap has > 1 node:
   - Extract two minimum nodes L and R
   - Create internal node with freq $= f(L) + f(R)$; L = left child, R = right child
   - Insert new node back into heap
4. Remaining node = root of Huffman tree

Assign codes: left edge = `0`, right edge = `1`; code for symbol = path from root to leaf.

Example:

Char	Freq	Code
A	5	0
B	2	100
C	1	101
D	3	11

Encoded size $= \sum(\text{freq} \times \text{code length})$ **vs.** fixed-width: n symbols needs $\lceil \log_2 n \rceil$ bits each.

Coin Change (Greedy)

Always pick the largest denomination that fits the remaining amount.

Only optimal for canonical coin systems (e.g., US coins). Fails for arbitrary denominations.

Bin Packing

Pack items of sizes ≤ 1 into bins of capacity 1.

- **First Fit**: place in first bin with room
- **Best Fit**: place in bin with least remaining space (still fits)
- **First Fit Decreasing (FFD)**: sort items descending, then First Fit — best approximation

Number Conversions

Decimal $\to$ **Binary**: repeatedly divide by 2, record remainders bottom-up.

Binary $\to$ **Decimal**: $\sum b_i \times 2^i$ (right to left, 0-indexed).

Decimal $\to$ **Hex**: divide by 16; **Hex** $\to$ **Bin**: each hex digit = 4 bits.